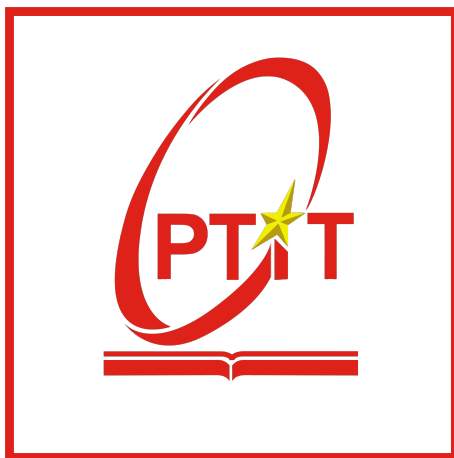


HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO

**CHUYÊN ĐỀ CÔNG NGHỆ
PHẦN MỀM**

Giảng viên hướng dẫn: TS. Nguyễn Duy Phương

Nhóm môn học: 01

Sinh viên thực hiện: Vũ Minh Thành

Mã sinh viên: B22DCCN802

Hà Nội, 2026

Mục lục

1	TỔNG QUAN BÀI TOÁN ĐỐI SÁNH XÂU	3
1.1	Giới thiệu	3
1.2	Bài toán đối sánh xâu	3
1.3	Đầu vào và đầu ra của bài toán	4
2	PHÂN LOẠI CÁC THUẬT TOÁN ĐỐI SÁNH MẪU	5
2.1	Nhóm thuật toán tìm kiếm mẫu từ trái sang phải	5
2.2	Nhóm thuật toán tìm kiếm mẫu từ phải sang trái	6
2.3	Các thuật toán tìm kiếm mẫu từ vị trí xác định	6
2.4	Nhóm thuật toán tìm kiếm mẫu từ vị trí bất kỳ	7
3	TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI	8
3.1	Thuật toán Brute Force	8
3.1.1	Đặc điểm chính	8
3.1.2	Mô tả thuật toán	8
3.1.3	Cài đặt bằng C++	9
3.1.4	Kiểm nghiệm thuật toán	10
3.2	Thuật toán Tìm kiếm bằng Ô-tô-mát (Automaton)	10
3.2.1	Đặc điểm chính	10
3.2.2	Mô tả thuật toán	11
3.2.3	Cài đặt bằng C++	11
3.2.4	Kiểm nghiệm thuật toán	13
3.3	Thuật toán Knuth-Morris-Pratt (KMP)	16
3.3.1	Tiền xử lý (xây dựng mảng LPS)	16
3.3.2	Quá trình tìm kiếm	18
3.3.3	Quá trình dịch chuyển	18
3.3.4	Lặp lại và kết thúc	19
3.3.5	Cài đặt thuật toán bằng C++	19
3.4	Thuật toán Karp-Rabin	21
3.4.1	Thuật toán	21
3.4.2	Xác định giá trị băm	22
3.4.3	Cài đặt thuật toán bằng C++	22

3.4.4	Độ phức tạp của thuật toán	24
3.5	Thuật toán Shift Or	25
3.5.1	Đặc điểm chính	25
3.5.2	Mô tả thuật toán	25
3.5.3	Cài đặt thuật toán bằng C++	26
3.5.4	Đánh giá độ phức tạp của thuật toán	28
3.5.5	Kiểm nghiệm thuật toán	28
3.6	Thuật toán Morriss-Pratt	29
3.6.1	Đặc điểm chính	29
3.6.2	Mô tả thuật toán	30
3.6.3	Cài đặt bằng C++	31
3.6.4	Kiểm nghiệm thuật toán	33
3.7	Thuật toán Apostolico-Crochemore	34
3.7.1	Đặc điểm chính	34
3.7.2	Mô tả thuật toán	35
3.7.3	Cài đặt bằng C++	36
3.7.4	Kiểm nghiệm thuật toán	37
3.8	Thuật toán Not So Naïve	39
3.8.1	Đặc điểm chính	39
3.8.2	Mô tả thuật toán	39
3.8.3	Cài đặt bằng C++	39
3.8.4	Kiểm nghiệm thuật toán	40
4	TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI	43
4.1	Thuật toán Boyer-Moore	43
4.1.1	Tiền xử lý	43
4.1.2	Tìm kiếm	44
4.1.3	Dịch chuyển mẫu (Shifting the Pattern)	45
4.1.4	Lặp lại quá trình so khớp	45
4.1.5	Điều kiện kết thúc (Termination)	45
4.1.6	Cài đặt thuật toán bằng C++	45
4.2	Thuật toán Turbo Boyer-Moore (Turbo-BM)	47
4.2.1	Đặc điểm chính	47
4.2.2	Mô tả thuật toán	48
4.2.3	Cài đặt bằng C++	49
4.2.4	Kiểm nghiệm thuật toán	53
4.3	Thuật toán Quick Search	54
4.3.1	Đặc điểm chính	54
4.3.2	Mô tả thuật toán	54

Chương 1

TỔNG QUAN BÀI TOÁN ĐỐI SÁNH XÂU

1.1 Giới thiệu

Đối sánh xâu (String Matching) là một trong những chủ đề nền tảng và quan trọng trong lĩnh vực xử lý văn bản và khoa học máy tính. Các thuật toán đối sánh xâu được xem như các thành phần cơ sở, được triển khai trong nhiều hệ thống thực tế, bao gồm cả các hệ điều hành và công cụ xử lý dữ liệu.

Không chỉ dừng lại ở xử lý văn bản, các thuật toán này còn đóng vai trò quan trọng trong nhiều lĩnh vực khác nhau như xử lý ảnh, xử lý ngôn ngữ tự nhiên (NLP), tin sinh học (Bioinformatics), cũng như thiết kế và tối ưu hóa phần mềm.

1.2 Bài toán đối sánh xâu

Bài toán đối sánh xâu (String Matching Problem) được hiểu là quá trình tìm kiếm một hoặc nhiều chuỗi con (pattern) xuất hiện trong một chuỗi văn bản (text), có thể có độ dài rất lớn.

Ký hiệu:

- Xâu mẫu (pattern) ký hiệu là:

$$X = (x_0, x_1, \dots, x_{m-1})$$

với độ dài m

- Văn bản (text) ký hiệu là:

$$Y = (y_0, y_1, \dots, y_{n-1})$$

với độ dài n

- Cả hai xâu được xây dựng trên một bảng chữ cái hữu hạn (alphabet) ký hiệu:

$$\Sigma$$

với kích thước $|\Sigma| = \sigma$.

Ví dụ, một xâu nhị phân trong mật mã học cũng có thể được xem là một pattern. Tương tự, một chuỗi ký tự như “ABD” có thể biểu diễn các chuỗi khác nhau trong ứng dụng thực tế như phân tích dữ liệu hoặc xử lý ngôn ngữ.

1.3 Đầu vào và đầu ra của bài toán

Input:

- Xâu mẫu $X = (x_0, x_1, \dots, x_{m-1})$, độ dài m .
- Văn bản $Y = (y_0, y_1, \dots, y_{n-1})$, độ dài n .

Output:

- Tất cả các vị trí xuất hiện của X trong Y .

Chương 2

PHÂN LOẠI CÁC THUẬT TOÁN ĐỐI SÁNH MẪU

Các thuật toán đối sánh mẫu có thể được phân loại thành nhiều nhóm khác nhau dựa trên chiến lược tìm kiếm. Trong phạm vi tài liệu này, các thuật toán được chia thành 4 nhóm chính.

2.1 Nhóm thuật toán tìm kiếm mẫu từ trái sang phải

Các thuật toán trong nhóm này thực hiện so khớp ký tự theo hướng từ trái sang phải của văn bản.

- Thuật toán Brute Force
- Search with an automaton
- Thuật toán Karp-Rabin
- Thuật toán Shift Or
- Thuật toán Morris-Pratt
- Thuật toán Knuth-Morris-Pratt
- Thuật toán Apostolico-Crochemore
- Thuật toán Not So Naïve

2.2 Nhóm thuật toán tìm kiếm mẫu từ phải sang trái

Khác với nhóm thuật toán duyệt từ trái sang phải, các thuật toán trong nhóm này thực hiện quá trình so khớp bắt đầu từ ký tự ngoài cùng bên phải của xâu mẫu. Cách tiếp cận này giúp phát hiện sự không khớp sớm ở phần cuối của mẫu, từ đó cho phép dịch chuyển cửa sổ tìm kiếm một cách hiệu quả hơn.

Nhờ đặc điểm này, nhóm thuật toán duyệt từ phải sang trái thường đạt hiệu năng cao trong thực tế, đặc biệt đối với các văn bản có độ dài lớn và bảng chữ cái có kích thước rộng.

Các thuật toán tiêu biểu trong nhóm này bao gồm:

- Thuật toán Boyer-Moore
- Thuật toán Turbo-BM
- Thuật toán Quick Search
- Thuật toán Tuned-Boyer-Moore
- Thuật toán Zhu-Takaoka
- Thuật toán Berry-Ravindran
- Thuật toán Apostolico-Giancarlo

2.3 Các thuật toán tìm kiếm mẫu từ vị trí xác định

Khác với các nhóm thuật toán trước, nhóm thuật toán tìm kiếm mẫu từ vị trí xác định không thực hiện so khớp trên toàn bộ văn bản một cách tuần tự, mà thay vào đó bắt đầu quá trình tìm kiếm từ các vị trí đã được xác định trước hoặc được tính toán dựa trên các đặc trưng của mẫu và văn bản.

Cách tiếp cận này giúp giảm không gian tìm kiếm, đặc biệt trong các bài toán có điều kiện ràng buộc về vị trí xuất hiện của mẫu hoặc khi văn bản có cấu trúc đặc biệt.

Các thuật toán trong nhóm này thường kết hợp với các kỹ thuật tiền xử lý để xác định các vị trí tiềm năng, từ đó thực hiện so khớp chi tiết tại các vị trí đó thay vì duyệt toàn bộ văn bản.

Một số thuật toán tiêu biểu trong nhóm này bao gồm:

- Thuật toán Colussi
- Thuật toán Skip Search

- Thuật toán Alpha Skip Search

2.4 Nhóm thuật toán tìm kiếm mẫu từ vị trí bất kỳ

Nhóm thuật toán tìm kiếm mẫu từ vị trí bất kỳ là nhóm cơ bản nhất trong bài toán đối sánh xâu, trong đó không có ràng buộc về vị trí xuất hiện của mẫu trong văn bản. Thuật toán sẽ thực hiện việc tìm kiếm toàn bộ các vị trí có thể trong văn bản để xác định sự xuất hiện của xâu mẫu.

Cách tiếp cận này thường được sử dụng trong các bài toán tổng quát, khi không có thông tin trước về vị trí của mẫu. Do đó, các thuật toán trong nhóm này thường có chiến lược duyệt tuần tự trên toàn bộ văn bản, kết hợp với các kỹ thuật tối ưu hóa để giảm số lần so khớp không cần thiết.

Đây là nhóm thuật toán nền tảng, từ đó phát triển nhiều kỹ thuật tối ưu hóa khác nhau trong các nhóm thuật toán nâng cao.

- Thuật toán Hospool
- Thuật toán Smith
- Thuật toán Raita

Chương 3

TÌM KIẾM MẪU TỪ TRÁI SANG PHẢI

3.1 Thuật toán Brute Force

3.1.1 Đặc điểm chính

- Không cần pha tiền xử lý.
- Bộ nhớ sử dụng là hằng số $O(1)$.
- Độ phức tạp thời gian pha tìm kiếm là $O(nm)$ trong trường hợp xấu nhất.

3.1.2 Mô tả thuật toán

Thuật toán Brute Force kiểm tra tất cả vị trí trong đoạn văn bản từ vị trí 0 đến $n - m$, xem liệu mẫu có khớp tại vị trí đó hay không. Tại mỗi vị trí j , thuật toán so sánh lần lượt từng ký tự của mẫu x với các ký tự tương ứng trong văn bản y . Sau mỗi lần kiểm tra, mẫu được dịch sang phải 1 vị trí.

Cụ thể, quá trình tìm kiếm diễn ra như sau:

1. Bắt đầu từ vị trí đầu tiên của văn bản và trượt cửa sổ mẫu qua toàn bộ văn bản.
2. Tại mỗi vị trí, so sánh lần lượt các ký tự của mẫu với các ký tự tương ứng trong văn bản.
3. Nếu phát hiện sự không khớp, dịch cửa sổ mẫu sang phải một vị trí.
4. Lặp lại bước 2 và 3 cho đến khi cửa sổ mẫu đạt đến vị trí cuối cùng có thể trong văn bản.
5. Nếu tất cả các ký tự trong mẫu khớp hoàn toàn với các ký tự tương ứng trong văn bản, ghi lại vị trí bắt đầu của lần khớp đó.

6. Dịch cửa sổ mẫu sang phải một vị trí và tiếp tục quá trình so sánh.

3.1.3 Cài đặt bằng C++

```
#include <iostream>
#include <vector>
using namespace std;

// Hàm tìm kiếm chuỗi mẫu x trong chuỗi y
vector<int> bruteForce(string y, string x) {
    vector<int> result;

    int n = y.size();
    int m = x.size();

    // Duyệt từng vị trí có thể
    for (int i = 0; i <= n - m; i++) {
        int j = 0;

        // So sánh từng ký tự
        while (j < m && y[i + j] == x[j]) {
            j++;
        }

        // Nếu khớp toàn bộ pattern
        if (j == m) {
            result.push_back(i);
        }
    }

    return result;
}

int main() {
    string y = "ababcabcabababd";
    string x = "ababd";

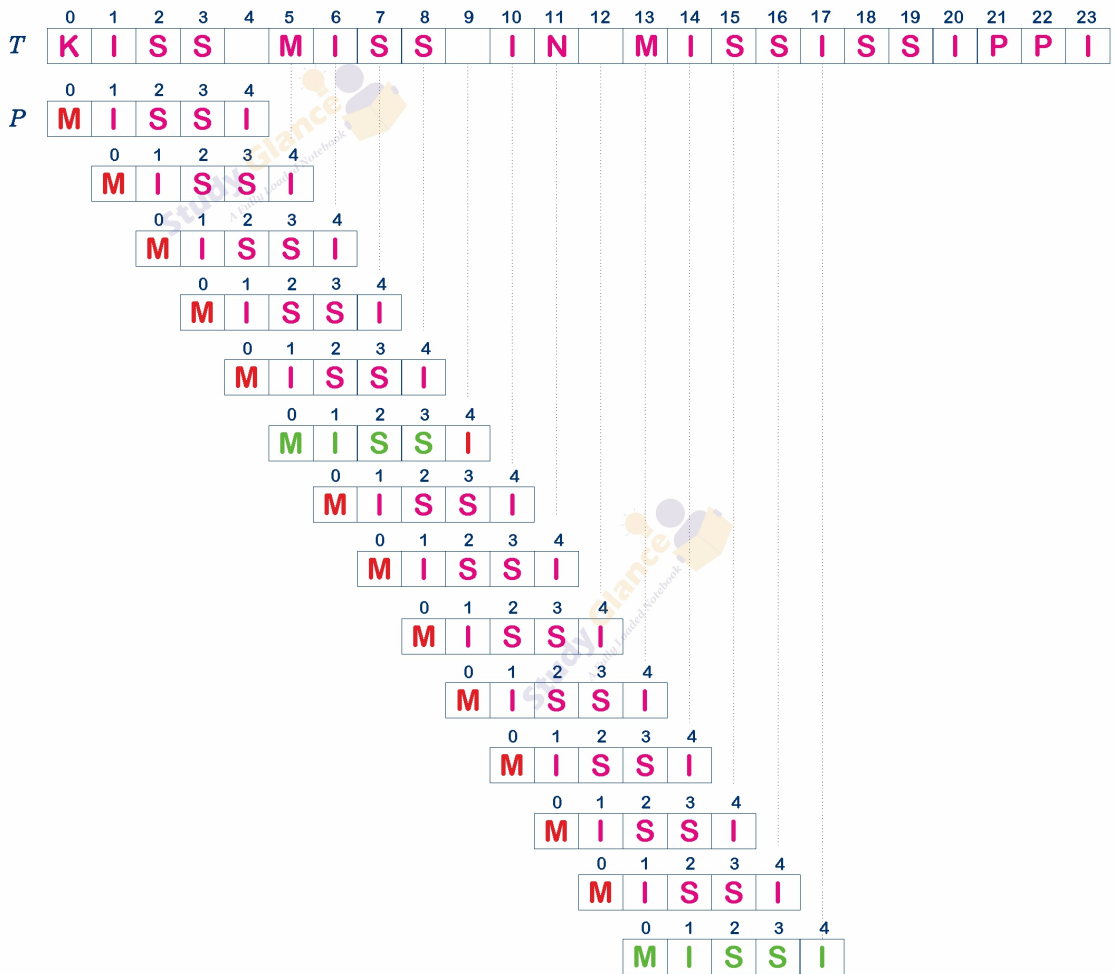
    vector<int> pos = bruteForce(y, x);

    cout << "Vi tri xuat hien: ";
    for (int p : pos) {
```

}

3.1.4 Kiểm nghiệm thuật toán

Giả sử ta có hai râu ký tự y (văn bản) và x (mẫu). Ta tiến hành tìm kiếm râu x trong râu y theo từng bước minh họa dưới đây.



3.2 Thuật toán Tìm kiếm bằng Ô-tô-mát (Automaton)

3.2.1 Đặc điểm chính

Thuật toán Tìm kiếm bằng Ô-tô-mát sử dụng một ô-tô-mát hữu hạn tất định (DFA) được xây dựng từ mẫu x để duyệt văn bản y chỉ trong một lượt, không cần quay lui. Đây là một trong những thuật toán tìm kiếm xâu hiệu quả nhất về số lần kiểm tra ký tự trong pha tìm kiếm. Các đặc điểm nổi bật của thuật toán bao gồm:

- Pha tiền xử lý xây dựng ô-tô-mát $\mathcal{A}(x)$ trong thời gian $O(m + |\Sigma|)$ và không gian $O(m \cdot |\Sigma|)$.
- Pha tìm kiếm có độ phức tạp thời gian $O(n)$ nếu ô-tô-mát được lưu trong bảng truy cập trực tiếp; $O(n \log |\Sigma|)$ trong trường hợp ngược lại.
- Thực hiện đúng n lần kiểm tra ký tự văn bản trong pha tìm kiếm.

3.2.2 Mô tả thuật toán

Tìm kiếm xâu x bằng ô-tô-mát bắt đầu bằng việc xây dựng ô-tô-mát hữu hạn tất định tối thiểu (DFA) $\mathcal{A}(x)$ nhận dạng ngôn ngữ Σ^*x . Ô-tô-mát này được định nghĩa là bộ bốn thành phần $\mathcal{A}(x) = (Q, q_0, T, E)$, trong đó:

- Q là tập các trạng thái, được đánh nhãn theo độ dài tiền tố tương ứng, từ 0 đến m .
- q_0 là trạng thái khởi đầu.
- T là tập các trạng thái kết thúc (terminal).
- E là tập các chuyển trạng thái; các chuyển còn thiếu đều dẫn về trạng thái khởi đầu 0.

Sau khi xây dựng xong $\mathcal{A}(x)$, pha tìm kiếm hoạt động như sau: văn bản y được phân tích tuần tự bằng $\mathcal{A}(x)$, bắt đầu từ trạng thái khởi đầu q_0 . Mỗi khi trạng thái kết thúc được đạt tới, một lần xuất hiện của x trong y được ghi nhận.

3.2.3 Cài đặt bằng C++

```
#include <iostream>
#include <vector>
#include <string>
#include <cstring>
using namespace std;

const int ASIZE = 256;

// Xây dựng bảng chuyển trạng thái của DFA
// aut[s][c] = trạng thái tiếp theo từ trạng thái s với ký tự c
void preAut(const string& x, int m,
            vector<vector<int>>& aut) {
    for (int i = 0; i < m; ++i) {
        // Sao chép chuyển trạng thái từ trạng thái hiện tại
```

```

        for (int c = 0; c < ASIZE; ++c)
            aut[i + 1][c] = aut[i][c];
        // Thiết lập chuyển đúng theo ký tự mẫu
        aut[i + 1][(unsigned char)x[i]] = i + 1;
        // Sửa lại các chuyển không khớp dựa trên hàm thất bại
        // (đã được mã hóa ngầm trong quá trình sao chép)
    }
}

// Tìm tất cả vị trí xuất hiện của x trong y
vector<int> AUT(const string& x, const string& y) {
    int m = (int)x.size();
    int n = (int)y.size();
    vector<int> results;

    // Khởi tạo bảng chuyển trạng thái kích thước (m+1) x ASIZE
    vector<vector<int>> aut(m + 1, vector<int>(ASIZE, 0));

    /* Preprocessing */
    preAut(x, m, aut);

    /* Searching */
    int state = 0;
    for (int j = 0; j < n; ++j) {
        state = aut[state][(unsigned char)y[j]];
        if (state == m)
            results.push_back(j - m + 1); // OUTPUT
    }

    return results;
}

int main() {
    string y = "ababcabcbabababd";
    string x = "abab";

    vector<int> pos = AUT(x, y);

    cout << "Vi tri xuat hien: ";
    for (int p : pos)

```

```

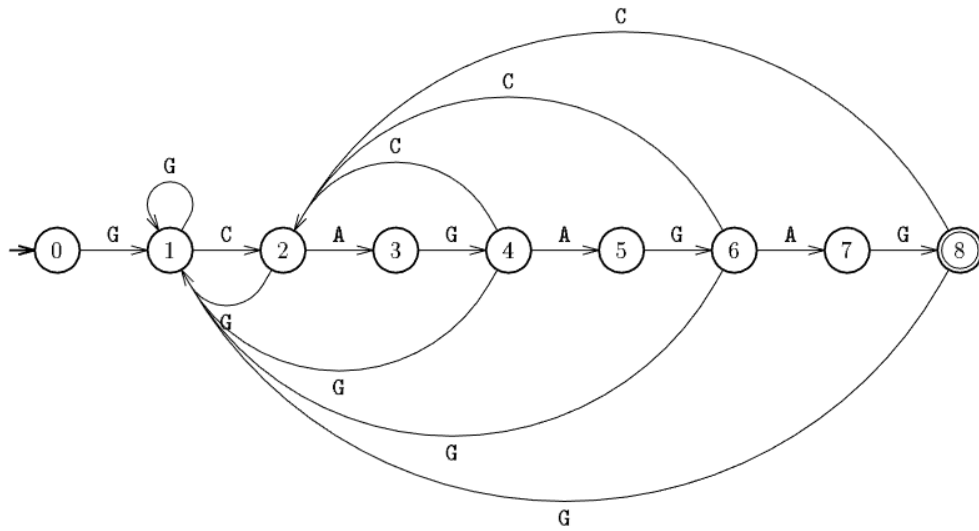
    cout << p << " ";
    cout << endl;

    return 0;
}

```

3.2.4 Kiểm nghiệm thuật toán

Giả sử ta có mẫu $x = \text{abab}$ và văn bản $y = \text{ababcabcbabababd}$. Ô-tô-mát $\mathcal{A}(x)$ được xây dựng với $m + 1 = 5$ trạng thái, đánh số từ 0 đến 4, trong đó trạng thái 4 là trạng thái kết thúc. Pha tìm kiếm duyệt qua từng ký tự của y , chuyển trạng thái theo bảng $\mathcal{A}(x)$ và ghi nhận vị trí khớp mỗi khi đạt trạng thái kết thúc.



Các trạng thái được đánh nhãn theo độ dài của tiền tố mà chúng đại diện. Các chuyển trạng thái không được định nghĩa đều dẫn về trạng thái khởi đầu 0.

Pha tìm kiếm

Trạng thái bắt đầu là 0.

y

G	C	A	T	C	G	C	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1

y G **C** A T C G C A G A G A G T A T A C A G T A C G
2

y G C **A** T C G C A G A G A G T A T A C A G T A C G
3

y G C A **T** C G C A G A G A G T A T A C A G T A C G
0

y G C A T **C** G C A G A G A G T A T A C A G T A C G
0

y G C A T C **G** C A G A G A G T A T A C A G T A C G
1

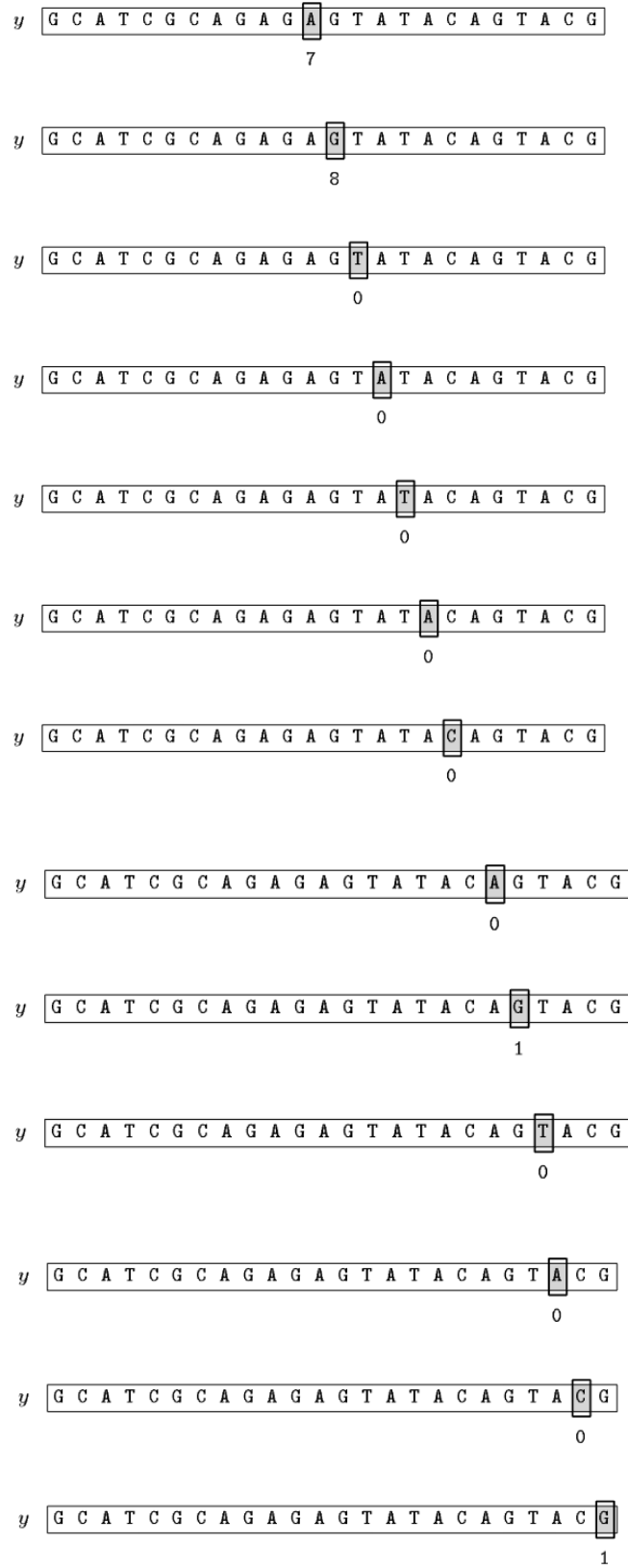
y G C A T C G **C** A G A G A G T A T A C A G T A C G
2

y G C A T C G C **A** G A G A G T A T A C A G T A C G
3

y G C A T C G C A **G** A G A G T A T A C A G T A C G
4

y G C A T C G C A G **A** G A G T A T A C A G T A C G
5

y G C A T C G C A G A **G** A G T A T A C A G T A C G
6



Thuật toán sử dụng 24 phép so sánh chuỗi trong ví dụ trên.

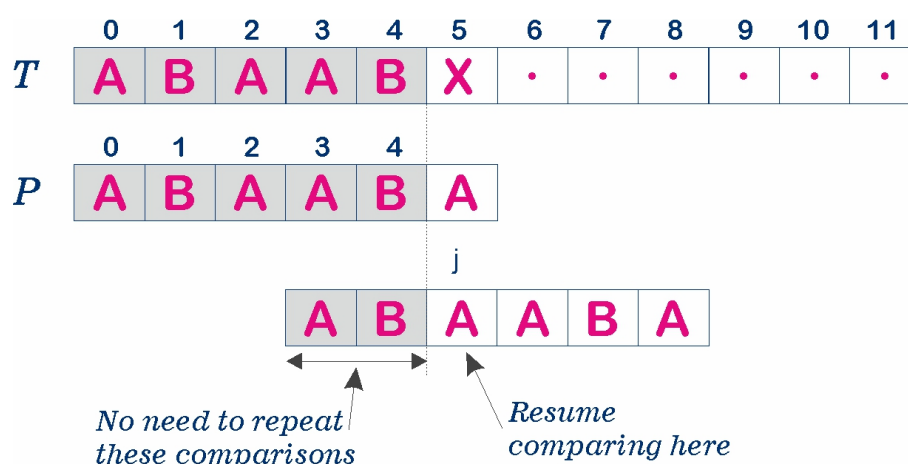
3.3 Thuật toán Knuth-Morris-Pratt (KMP)

Thuật toán đối sánh mẫu Knuth-Morris-Pratt (KMP) là một phương pháp tìm kiếm chuỗi hiệu quả, được phát triển bởi Donald Knuth, James H. Morris và Vaughan Pratt vào năm 1970. Thuật toán này được sử dụng để tìm tất cả các vị trí xuất hiện của một xâu mẫu (pattern) trong một văn bản (text) mà không cần phải so sánh lại từng ký tự một cách lặp lại như trong phương pháp Brute Force.

Thuật toán KMP thực hiện việc so sánh xâu mẫu với văn bản theo hướng từ trái sang phải, nhưng khác với Brute Force ở chỗ nó dịch chuyển xâu mẫu một cách thông minh hơn khi xảy ra sự không khớp.

Khi xảy ra một vị trí không khớp, thuật toán sẽ xác định khoảng dịch chuyển lớn nhất có thể mà không làm mất thông tin đã so khớp trước đó. Cụ thể, thuật toán tìm **tiền tố dài nhất của đoạn đã xét trong pattern đồng thời cũng là hậu tố của chính đoạn đó**, từ đó quyết định vị trí dịch chuyển tiếp theo của mẫu.

Nói cách khác, nếu đang so khớp đến vị trí j và xảy ra sai khác, KMP sẽ sử dụng mảng tiền xử lý (lps - longest prefix suffix) để xác định độ dài của tiền tố dài nhất cũng là hậu tố của đoạn $P[0..j]$, nhằm tránh việc so sánh lại các ký tự đã biết là trùng khớp.



3.3.1 Tiền xử lý (xây dựng mảng LPS)

Ý tưởng cốt lõi của thuật toán KMP là tiền xử lý xâu mẫu để xây dựng mảng LPS (Longest Prefix Suffix). Mảng này lưu độ dài của tiền tố dài nhất đồng thời cũng là hậu tố của từng đoạn con của xâu mẫu.

Việc xây dựng mảng LPS giúp thuật toán xác định vị trí dịch chuyển hợp lý khi xảy ra sai khác, từ đó tránh việc so sánh lại các ký tự đã biết.

Thuật toán xây dựng LPS:

- Khởi tạo $lps[0] = 0$ vì một ký tự đơn không có tiền tố hoặc hậu tố thực sự.
- Sử dụng hai biến: len (độ dài tiền tố - hậu tố hiện tại) và i (duyệt pattern), với $len = 0, i = 1$.

- Trong khi $i < m$:

- Nếu $pattern[len] = pattern[i]$:

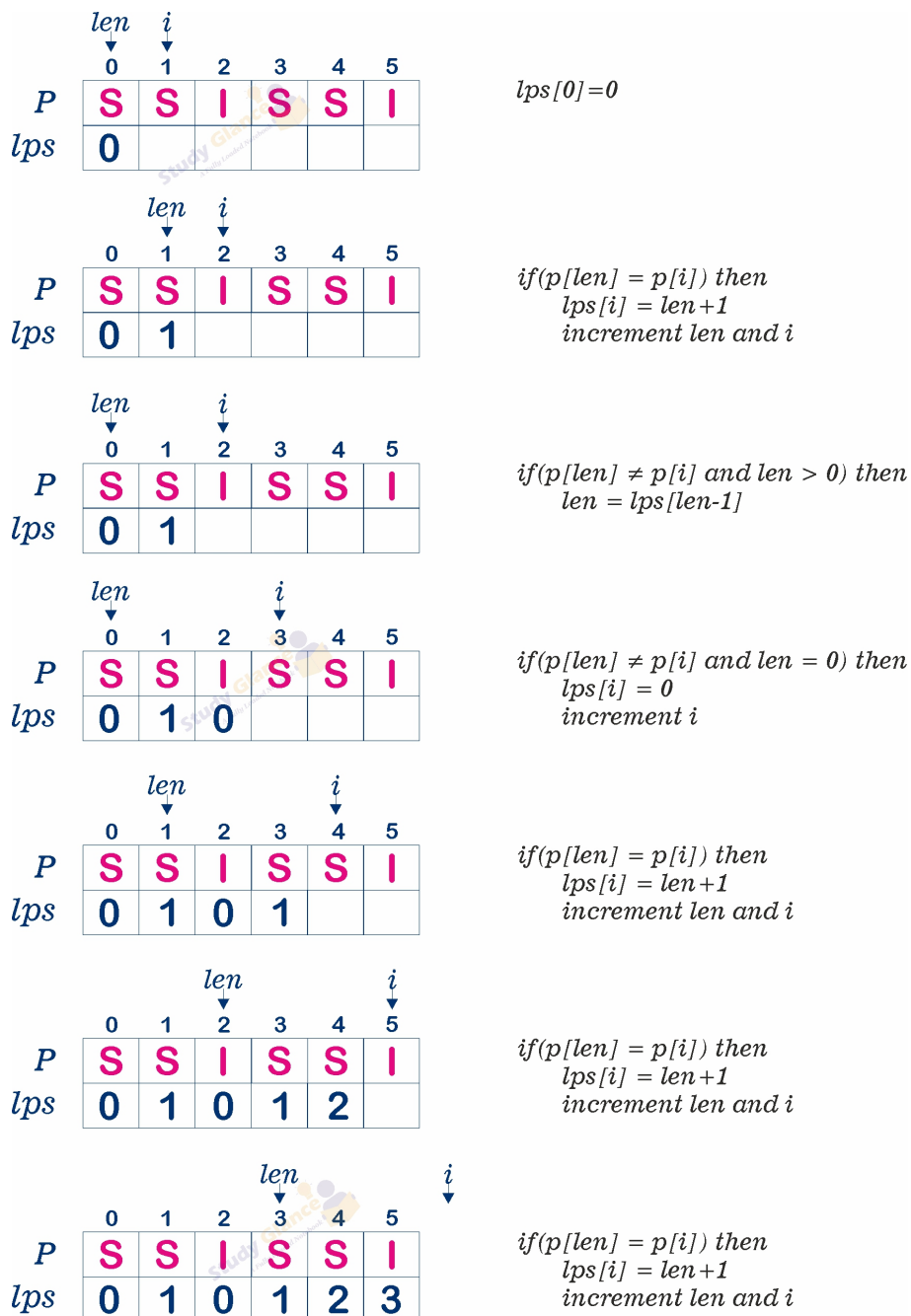
$$lps[i] = len + 1, \quad len = len + 1, \quad i = i + 1$$

- Nếu không khớp và $len \neq 0$:

$$len = lps[len - 1]$$

- Nếu không khớp và $len = 0$:

$$lps[i] = 0, \quad i = i + 1$$



3.3.2 Quá trình tìm kiếm

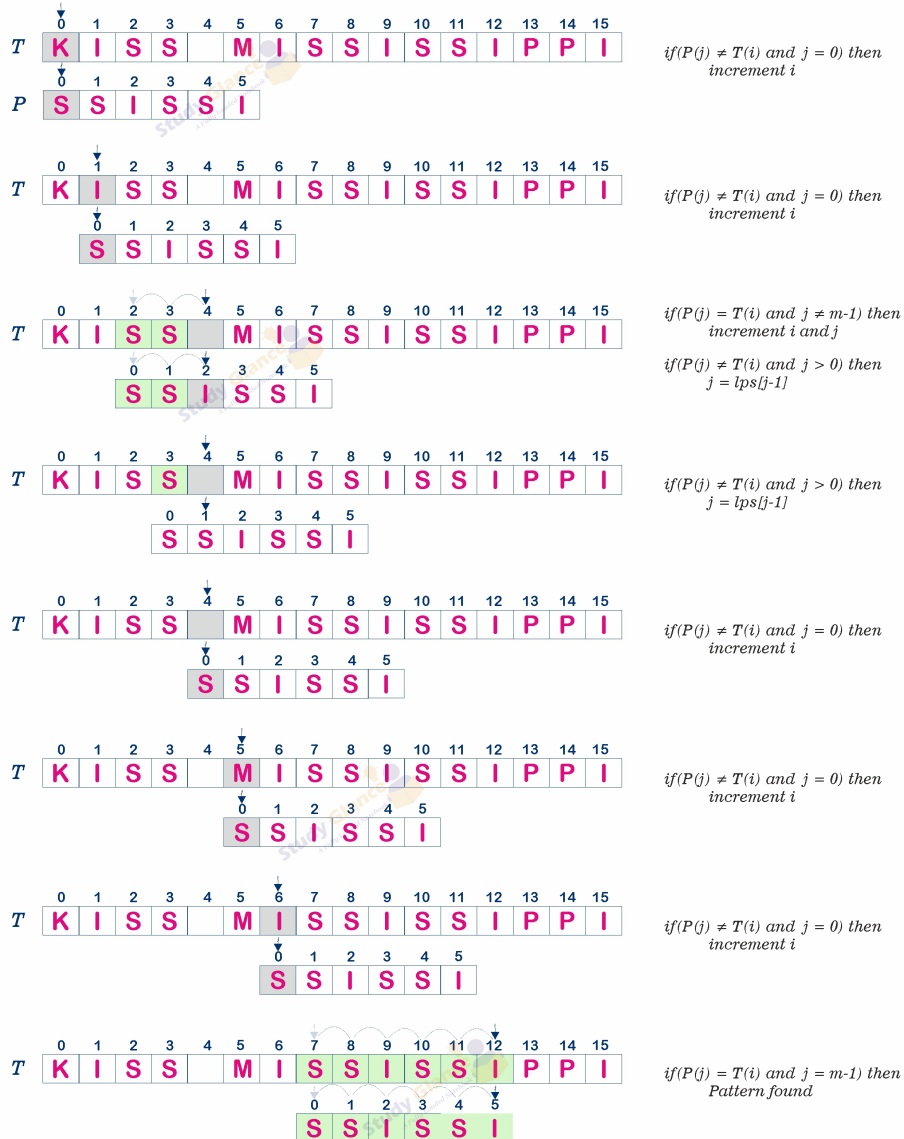
Sau khi hoàn tất bước tiền xử lý, thuật toán bắt đầu tìm kiếm trong văn bản.

- Căn chỉnh mẫu với đầu văn bản.
- So sánh ký tự của mẫu và văn bản từ trái sang phải.
- Nếu toàn bộ ký tự khớp, ghi nhận một lần xuất hiện của mẫu.

3.3.3 Quá trình dịch chuyển

Khi xảy ra sai khác tại vị trí j :

- Nếu $pattern[j] = text[i]$, tăng cả i và j .
- Nếu $j = m$, một lần khớp được tìm thấy, sau đó đặt $j = lps[j - 1]$.
- Nếu xảy ra sai khác và $j \neq 0$, cập nhật $j = lps[j - 1]$ (không tăng i).
- Nếu xảy ra sai khác và $j = 0$, tăng i .



3.3.4 Lặp lại và kết thúc

Thuật toán tiếp tục quá trình so sánh cho đến khi:

- Duyệt hết văn bản, hoặc
- Tìm được tất cả các vị trí xuất hiện của mẫu trong văn bản.

3.3.5 Cài đặt thuật toán bằng C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// Hàm xây dựng mảng LPS
vector<int> buildLPS(string pattern) {
    int m = pattern.size();
    vector<int> lps(m, 0);

    int len = 0; // độ dài prefix suffix
    int i = 1;

    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        }
        else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    return lps;
}
```

```

// Hàm tìm kiếm KMP
vector<int> KMP(string text, string pattern) {
    vector<int> result;

    int n = text.size();
    int m = pattern.size();

    vector<int> lps = buildLPS(pattern);

    int i = 0; // index text
    int j = 0; // index pattern

    while (i < n) {
        if (text[i] == pattern[j]) {
            i++;
            j++;
        }

        if (j == m) {
            result.push_back(i - j);
            j = lps[j - 1];
        }
        else if (i < n && text[i] != pattern[j]) {
            if (j != 0)
                j = lps[j - 1];
            else
                i++;
        }
    }

    return result;
}

int main() {
    string text = "ababcabababd";
    string pattern = "ababd";

    vector<int> pos = KMP(text, pattern);

    cout << "Vi tri xuat hien: ";

```

```

    for (int x : pos) {
        cout << x << " ";
    }

    return 0;
}

```

3.4 Thuật toán Karp-Rabin

Thuật toán Rabin-Karp là một thuật toán tìm kiếm chuỗi sử dụng kỹ thuật băm (hashing) để so khớp chuỗi mẫu trong văn bản. Thuật toán này được phát triển bởi Richard M. Karp và Michael O. Rabin vào năm 1987.

Đối với một văn bản T có độ dài n và một chuỗi mẫu P có độ dài m , độ phức tạp trung bình của thuật toán là $O(n + m)$. Tuy nhiên, trong trường hợp xấu nhất, độ phức tạp của thuật toán có thể trở thành $O(n \cdot m)$, tương đương với thuật toán Brute Force do xảy ra nhiều va chạm (collision) trong quá trình băm.

Mặc dù thuật toán Knuth-Morris-Pratt (KMP) thường hiệu quả hơn trong bài toán tìm kiếm chuỗi đơn lẻ, Rabin-Karp lại được sử dụng rộng rãi trong các hệ thống phát hiện đạo văn (plagiarism detection), nơi cần so khớp nhiều đoạn văn bản với nhau trong thời gian ngắn bằng cách sử dụng giá trị băm để tăng tốc quá trình so sánh.

3.4.1 Thuật toán

Nếu xét phương pháp tiếp cận Brute Force, việc tìm kiếm một mẫu trong chuỗi mất thời gian $O(mn)$. Đặt câu hỏi: nếu có thể so sánh hai ký tự trong thời gian $O(1)$ như so sánh số nguyên thì sao?

Thuật toán Rabin-Karp giải quyết vấn đề này bằng cách sử dụng kỹ thuật băm (hashing), cho phép chuyển một chuỗi ký tự thành một giá trị số nguyên. Ý tưởng là biểu diễn chuỗi ký tự dưới dạng một số theo hệ cơ số B , trong đó B lớn hơn giá trị ASCII lớn nhất của các ký tự trong bảng chữ cái.

Sau đó, giá trị theo hệ cơ số B này được chuyển sang hệ cơ số 10 để thu được giá trị băm (hash value) của chuỗi. Việc này được thực hiện theo công thức:

$$\text{hash}(S) = s_0 \cdot B^{m-1} + s_1 \cdot B^{m-2} + \dots + s_{m-1} \cdot B^1 + s_m \cdot B^0$$

Trong công thức trên, $s(i)$ là giá trị ASCII của ký tự thứ i trong chuỗi s .

Nhờ cách biểu diễn này, mỗi chuỗi ký tự (trong điều kiện lý tưởng và không xảy ra va chạm hash) sẽ tương ứng với một giá trị băm duy nhất. Điều này giúp việc so sánh hai chuỗi có thể được thực hiện nhanh chóng thông qua việc so sánh giá trị hash thay vì so sánh từng ký tự.

3.4.2 Xác định giá trị băm

Mục tiêu chính khi cài đặt thuật toán là tính toán giá trị băm một cách hiệu quả. Đầu tiên, ta cần tính giá trị băm của xâu mẫu P . Sau đó, ta so sánh với giá trị băm của từng đoạn con có độ dài m trong xâu S .

Nếu ta tính trực tiếp giá trị băm cho mỗi đoạn con có độ dài m , độ phức tạp sẽ là $O(mn)$. Khi đó, cách tiếp cận này không còn ưu việt hơn so với thuật toán Brute Force.

Vì vậy, ta cần một phương pháp tối ưu hơn để giảm độ phức tạp xuống $O(n)$.

Giả sử $H(i)$ là giá trị băm của đoạn con có độ dài m trong chuỗi S , bắt đầu từ vị trí i . Khi đó ta có thể biểu diễn như sau:

$$\text{hash}(S) = s_0 \cdot B^{m-1} + s_1 \cdot B^{m-2} + \dots + s_{m-1} \cdot B^1 + s_m \cdot B^0$$

Với $m = 3$, ta có thể viết $H(0)$ và $H(1)$ thế này

$$H_0 = s_0 \cdot B^2 + s_1 \cdot B^1 + s_2$$

$$H_1 = s_1 \cdot B^2 + s_2 \cdot B^1 + s_3$$

Giờ ta có thể viết $H(1)$ biểu diễn bằng $H(0)$ như sau:

$$H_1 = ((s_0 \cdot B^2 + s_1 \cdot B^1 + s_2) - (s_0 \cdot B^2)) \cdot B + s_3$$

$$H_1 = (H_0 - S_0 \cdot B^2) \cdot B + s_3$$

Vậy nên, ta có công thức tổng quát:

$$H_i = (H(i-1) - S(i-1) \cdot B^{m-1}) \cdot B + s_{i+m-1}$$

Giờ đây ta có thể lấy giá trị băm của mỗi xâu con size m trong xâu S với độ phức tạp $O(n)$. Ta chỉ cần tìm giá trị băm của m ký tự đầu tiên. Các ký tự còn lại ta có thể xác định dựa trên thuật toán băm.

3.4.3 Cài đặt thuật toán bằng C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

typedef long long ll;
```

```

const ll B = 256;          // base (hệ cơ số)
const ll MOD = 1e9 + 7;    // modulo để tránh tràn số

// Hàm tính lũy thừa ( $B^{(m-1)}$ )
ll power(ll base, ll exp) {
    ll result = 1;
    for (ll i = 0; i < exp; i++)
        result = (result * base) % MOD;
    return result;
}

// Hàm Rabin-Karp
vector<int> rabinKarp(string text, string pattern) {
    vector<int> result;

    int n = text.size();
    int m = pattern.size();

    if (m > n) return result;

    ll hashP = 0; // hash của pattern
    ll hashT = 0; // hash của window text
    ll h = power(B, m - 1);

    // Tính hash ban đầu
    for (int i = 0; i < m; i++) {
        hashP = (B * hashP + pattern[i]) % MOD;
        hashT = (B * hashT + text[i]) % MOD;
    }

    // Duyệt từng cửa sổ
    for (int i = 0; i <= n - m; i++) {

        // Nếu hash khớp thì kiểm tra từng ký tự
        if (hashP == hashT) {
            bool match = true;
            for (int j = 0; j < m; j++) {
                if (text[i + j] != pattern[j]) {
                    match = false;
                    break;
                }
            }
        }
    }
}

```

```

        }
    }
    if (match) result.push_back(i);
}

// Cập nhật rolling hash
if (i < n - m) {
    hashT = (B * (hashT - text[i] * h) + text[i + m]) % MOD;

    if (hashT < 0)
        hashT += MOD;
}
}

return result;
}

int main() {
    string text = "ababcabcabababd";
    string pattern = "ababd";

    vector<int> pos = rabinKarp(text, pattern);

    cout << "Vi tri xuat hien: ";
    for (int i : pos) {
        cout << i << " ";
    }

    return 0;
}

```

3.4.4 Độ phức tạp của thuật toán

Trong cài đặt này, giá trị băm của xâu mẫu được tính trong thời gian $O(m)$, trong khi giá trị băm của văn bản được tính trong thời gian $O(n)$. Do đó, tổng độ phức tạp của bước khởi tạo là $O(m + n)$.

Tuy nhiên, trong quá trình tính toán, giá trị băm có thể bị tràn số (overflow). Để khắc phục vấn đề này, ta sử dụng phép toán modulo với một số lớn M , nhằm giới hạn phạm vi giá trị của hash.

Mặc dù vậy, việc sử dụng modulo có thể dẫn đến hiện tượng va chạm (collision), khi hai xâu khác nhau nhưng lại có cùng giá trị băm. Trong trường hợp này, ta cần

kiểm tra lại từng ký tự của xâu để xác nhận kết quả, điều này có thể làm tăng độ phức tạp lên $O(mn)$ trong trường hợp xấu nhất.

Để giảm thiểu va chạm, có thể sử dụng kỹ thuật rehashing với các giá trị cơ sở B và modulo M khác nhau. Tuy nhiên, trong một số trường hợp, việc kiểm tra lại từng ký tự vẫn có thể xảy ra, làm giảm hiệu quả của thuật toán.

Do đó, trong nhiều tình huống thực tế, thuật toán KMP vẫn được ưu tiên hơn do đảm bảo độ phức tạp ổn định và không phụ thuộc vào xác suất va chạm như Rabin-Karp.

3.5 Thuật toán Shift Or

3.5.1 Đặc điểm chính

Thuật toán Shift-Or (còn được gọi là Shift-And) là một thuật toán tìm kiếm chuỗi dựa trên phép toán bit (bitwise operations), được sử dụng để xác định sự xuất hiện của xâu mẫu trong văn bản một cách hiệu quả. Thuật toán này đặc biệt phù hợp khi độ dài của xâu mẫu không quá lớn và có thể được biểu diễn bằng các bit trong một từ máy.

Thuật toán hoạt động bằng cách biểu diễn trạng thái khớp của các ký tự trong mẫu dưới dạng các phép dịch bit và phép toán logic AND/OR. Nhờ đó, quá trình so khớp ký tự được thực hiện song song ở mức bit, giúp tăng tốc độ xử lý so với các phương pháp so sánh tuần tự truyền thống.

Thuật toán Shift-Or có các đặc điểm sau:

- Sử dụng các kỹ thuật thao tác bit (bitwise techniques) để thực hiện so khớp.
- Hiệu quả khi độ dài của xâu mẫu không vượt quá kích thước từ nhớ (word size) của máy tính.
- Giai đoạn tiền xử lý có độ phức tạp $O(m + \sigma)$ về cả thời gian và không gian, trong đó m là độ dài xâu mẫu và σ là kích thước bảng chữ cái.
- Giai đoạn tìm kiếm có độ phức tạp $O(n)$, không phụ thuộc vào kích thước bảng chữ cái hay độ dài xâu mẫu.
- Dễ dàng mở rộng để áp dụng cho bài toán tìm kiếm xâu xấp xỉ (approximate string matching).

3.5.2 Mô tả thuật toán

Thuật toán Shift Or sử dụng thao tác bit. Gọi R là mảng bit có kích thước m . Vector R_j là giá trị của mảng R sau khi ký tự $y[j]$ đã được xử lý. Vector này chứa thông tin về tất cả các tiền tố của x mà kết thúc tại vị trí j trong xâu, với $0 < i \leq m - 1$

Ta có công thức sau:

$$R_j[i] = \begin{cases} 0 & \text{if } x[0..i] = y[j - i..j] \\ 1 & \text{otherwise} \end{cases}$$

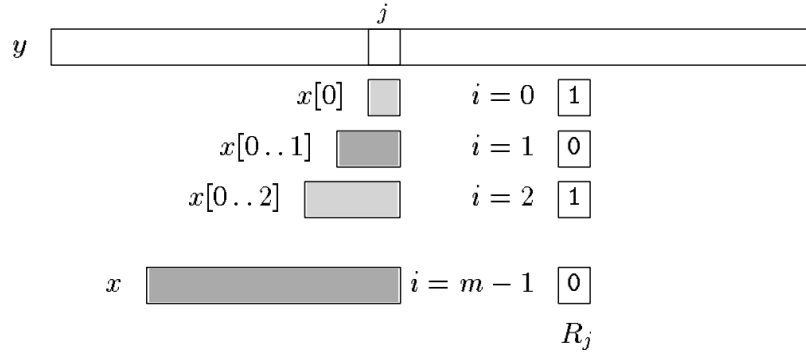
Vector R_{j+1} có thể được tính từ R_j như sau. Với mỗi vị trí mà $R_j[i] = 0$, ta có:

$$R_{j+1}[i + 1] = \begin{cases} 0 & \text{if } x[i + 1] = y[j + 1] \\ 1 & \text{otherwise} \end{cases}$$

và

$$R_{j+1}[0] = \begin{cases} 0 & \text{if } x[0] = y[j + 1] \\ 1 & \text{otherwise} \end{cases}$$

Nếu $R_{j+1}[m - 1] = 0$ thì một chuỗi so khớp sẽ được ghi nhận.



Chuyển từ R_j sang R_{j+1} có thể tính toán rất nhanh dựa theo: Với mỗi $c \in \Sigma$, gọi S_c là một vector bit có kích thước m thỏa mãn:

với $0 \leq i \leq m - 1$, $S_c[i] = 0$ khi và chỉ khi $x[i] = c$.

Mảng S_c chỉ ra các vị trí của của ký tự c trong mẫu x . Mỗi S_c có thể được tiền xử lý trước khi tìm kiếm. Và quá trình tính toán của R_{j+1} được giảm xuống còn 2 thao tác, **shift** và **or**:

$$R_{j+1} = \text{SHIFT}(R_j) \text{ OR } S_{y[j+1]}$$

Giả sử độ dài mẫu không vượt quá kích thước từ nhớ (machine word) của máy, giai đoạn tiền xử lý yêu cầu độ phức tạp thời gian và không gian là $O(m + \sigma)$.

Giai đoạn tìm kiếm có độ phức tạp thời gian $O(n)$, không phụ thuộc vào kích thước bảng chữ cái và độ dài mẫu.

3.5.3 Cài đặt thuật toán bằng C++

```
#include <iostream>
#include <vector>
```

```

#include <string>

using namespace std;

const int ASIZE = 256;

vector<int> shiftOr(const string& pattern, const string& text) {
    int m = pattern.length();
    int n = text.length();

    if (m > 31) {
        throw runtime_error("Pattern length must not exceed word size");
    }

    vector<unsigned int> mask(ASIZE, ~0);

    unsigned int state = ~0;
    unsigned int lim = 0;

    /*
     * Preprocessing phase
     */
    unsigned int bit = 1;

    for (int i = 0; i < m; ++i) {
        mask[(unsigned char)pattern[i]] &= ~bit;
        lim |= bit;
        bit <<= 1;
    }

    lim = ~(lim >> 1);

    /*
     * Searching phase
     */
    vector<int> occurrences;

    for (int j = 0; j < n; ++j) {
        state = (state << 1) | mask[(unsigned char)text[j]];
    }
}

```

```

        if (state < lim) {
            occurrences.push_back(j - m + 1);
        }
    }

    return occurrences;
}

int main() {
    string text = "ababcababababd";
    string pattern = "ababd";

    vector<int> result = shiftOr(pattern, text);

    for (int pos : result) {
        cout << "Pattern found at position: " << pos << endl;
    }

    return 0;
}

```

3.5.4 Đánh giá độ phức tạp của thuật toán

Giả sử độ dài mẫu không lớn hơn kích thước của từ nhớ máy tính.

- Độ phức tạp pha tiền xử lý là $O(m + \sigma)$.
- Độ phức tạp của pha tìm kiếm là $O(n)$.

3.5.5 Kiểm nghiệm thuật toán

Tính các vector S_c :

	S_A	S_C	S_G	S_T
G	1	1	0	1
C	1	0	1	1
A	0	1	1	1
G	1	1	0	1
A	0	1	1	1
G	1	1	0	1
A	0	1	1	1
G	1	1	0	1

Tính các vector R_j :

		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
		G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
0	G	0	1	1	1	1	0	1	1	0	1	0	1	0	1	1	1	1	1	0	1	1	1	0	
1	C	1	0	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
2	A	1	1	0	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
3	G	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	
4	A	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	
5	G	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	1	
6	A	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	1	
7	G	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	1	

Với $R_12[7] = 0$ tức là x được tìm thấy tại vị trí $12 - 8 + 1 = 5$.

3.6 Thuật toán Morriss-Pratt

3.6.1 Đặc điểm chính

Thuật toán Morris-Pratt là một trong những thuật toán đối sánh mẫu kinh điển, được đề xuất bởi James H. Morris và Vaughan R. Pratt vào năm 1970. Thuật toán được xây dựng nhằm cải thiện hiệu năng của phương pháp đối sánh tuần tự (Brute Force) bằng cách tránh việc so sánh lại các ký tự đã được kiểm tra trước đó.

Ý tưởng cốt lõi của thuật toán là sử dụng thông tin thu được từ các lần khớp trước để xác định vị trí dịch chuyển tiếp theo của mẫu mà không cần quay lui trên văn bản. Để thực hiện điều này, thuật toán xây dựng một bảng tiền xử lý, thường được gọi là bảng thất bại (failure function) hoặc bảng biên (border table), cho biết độ dài tiền tố dài nhất của mẫu đồng thời cũng là hậu tố của chuỗi con đang xét.

Trong quá trình tìm kiếm, khi xảy ra sai khớp, thuật toán không dịch mẫu từng ký tự như phương pháp vét cạn mà sử dụng bảng tiền xử lý để xác định trạng thái khớp phù hợp tiếp theo. Nhờ đó, số lượng phép so sánh được giảm đáng kể.

Thuật toán Morris-Pratt có các đặc điểm chính như sau:

- Thực hiện quá trình so sánh từ trái sang phải.
- Pha tiền xử lý có độ phức tạp thời gian và không gian là $O(m)$, với m là độ dài mẫu.
- Pha tìm kiếm có độ phức tạp thời gian $O(n + m)$, không phụ thuộc vào kích thước bảng chữ cái.
- Thực hiện nhiều nhất $2n - 1$ phép so sánh ký tự trong quá trình quét văn bản.
- Độ trễ (delay) được chặn trên bởi m .

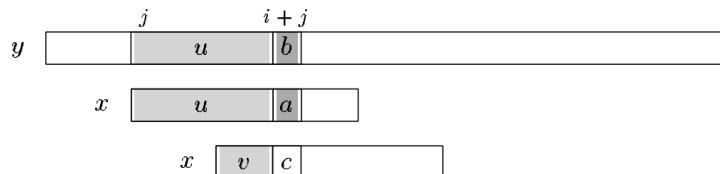
3.6.2 Mô tả thuật toán

Thuật toán Morris-Pratt được xây dựng dựa trên sự phân tích chi tiết thuật toán Brute Force, đặc biệt tập trung vào cách mà thuật toán này chưa tận dụng hiệu quả các thông tin thu được trong quá trình quét văn bản.

Khi phân tích chi tiết thuật toán Brute Force, có thể nhận thấy rằng độ dài dịch chuyển của mẫu có thể được cải thiện đồng thời lưu lại các phần của văn bản đã khớp với mẫu. Nhờ đó, số phép so sánh giữa các ký tự của mẫu và văn bản được giảm đáng kể, góp phần nâng cao tốc độ tìm kiếm.

Xét một lần thử đối sánh tại vị trí bắt đầu j trên chuỗi văn bản y , tức là khi cửa sổ đối sánh được đặt trên đoạn $y[j \dots j + m - 1]$. Giả sử phép sai khớp đầu tiên xảy ra giữa $x[i]$ và $y[i + j]$, với $0 < i < m$. Khi đó, ta có $x[0 \dots i - 1] = y[j \dots i + j - 1] = u$ và $a = x[i] \neq y[i + j] = b$. Nói cách khác, u là đoạn tiền tố của mẫu đã khớp hoàn toàn với đoạn tương ứng trong văn bản trước khi xảy ra sai khớp.

Khi thực hiện dịch chuyển mẫu, có thể kỳ vọng rằng một tiền tố v của mẫu sẽ khớp với một hậu tố của đoạn u trong văn bản. Tiền tố dài nhất thỏa mãn điều kiện này được gọi là biên (border) của u , tức là chuỗi xuất hiện đồng thời ở cả đầu và cuối của u . Từ đó, định nghĩa $mpNext[i]$ là độ dài của biên dài nhất của $x[0 \dots i - 1]$, với $0 < i \leq m$. Khi đó, sau mỗi lần dịch chuyển, quá trình so sánh có thể tiếp tục giữa ký tự $c = x[mpNext[i]]$ và $y[i + j] = b$ mà không bỏ sót bất kỳ lần xuất hiện nào của x trong y , đồng thời tránh việc quay lui trên văn bản. Giá trị $mpNext[0]$ được gán bằng -1 .



Bảng `mpNext` có thể được tính trước pha tìm kiếm với độ phức tạp thời gian và không gian $O(m)$ bằng cách áp dụng chính thuật toán tìm kiếm lên mẫu, xem như $x = y$.

Khi đó, pha tìm kiếm có thể được thực hiện với độ phức tạp thời gian $O(m + n)$. Trong quá trình tìm kiếm, thuật toán Morris-Pratt thực hiện nhiều nhất $2n - 1$ phép so sánh ký tự trên văn bản. Độ trễ của thuật toán (số phép so sánh tối đa đối với một ký tự văn bản) được chặn trên bởi m .

3.6.3 Cài đặt bằng C++

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

vector<int> computeMPNext(const string& pattern) {
    int m = pattern.length();

    vector<int> mpNext(m + 1);

    int i = 0;
    int j = -1;

    mpNext[0] = -1;

    while (i < m) {
        while (j > -1 && pattern[i] != pattern[j]) {
            j = mpNext[j];
        }

        ++i;
        ++j;

        mpNext[i] = j;
    }

    return mpNext;
}

vector<int> morrisPratt(const string& pattern, const string& text) {
```

```

    int m = pattern.length();
    int n = text.length();

    vector<int> mpNext = computeMPNext(pattern);

    vector<int> occurrences;

    int i = 0;
    int j = 0;

    while (j < n) {
        while (i > -1 && pattern[i] != text[j]) {
            i = mpNext[i];
        }

        ++i;
        ++j;

        if (i >= m) {
            occurrences.push_back(j - i);
            i = mpNext[i];
        }
    }

    return occurrences;
}

int main() {
    string text = "ababcababababd";
    string pattern = "ababd";

    vector<int> result = morrisPratt(pattern, text);

    for (int pos : result) {
        cout << "Pattern found at position: " << pos << endl;
    }

    return 0;
}

```

3.6.4 Kiểm nghiệm thuật toán

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$mpNext[i]$	-1	0	0	0	1	0	1	0	1

Pha tìm kiếm Lần 1:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1 2 3 4
 x G C A G A G A G
 Shift by 3 ($i - mpNext[i] = 3 - 0$)

Lần 2:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G
 Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 3:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G
 Shift by 1 ($i - mpNext[i] = 0 - -1$)

Lần 4:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1 2 3 4 5 6 7 8
 x G C A G A G A G
 Shift by 7 ($i - mpNext[i] = 8 - 1$)

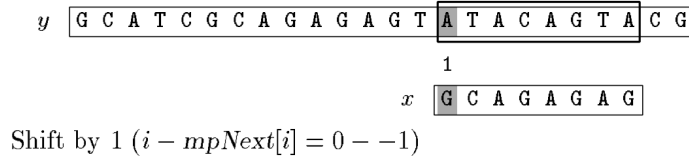
Lần 5:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G
 Shift by 1 ($i - mpNext[i] = 1 - 0$)

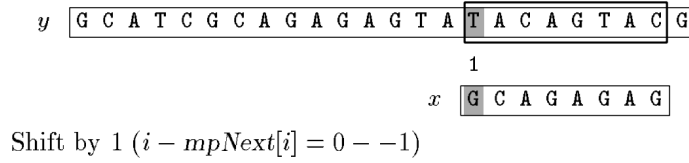
Lần 6:

y G C A T C G C A G A G A G T A T A C A G T A C G
 1
 x G C A G A G A G
 Shift by 1 ($i - mpNext[i] = 0 - -1$)

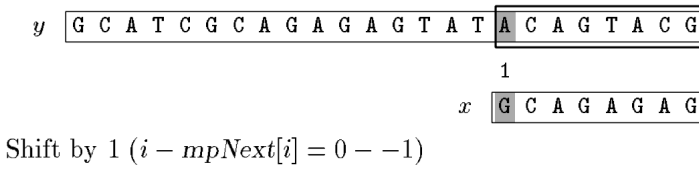
Lần 7:



Lần 8:



Lần 9:



Thuật toán Morris-Pratt thực hiện 19 lần so khớp chuỗi trong ví dụ trên.

3.7 Thuật toán Apostolico-Crochemore

3.7.1 Đặc điểm chính

Thuật toán Apostolico-Crochemore là một thuật toán tìm kiếm mẫu trong chuỗi, được thiết kế nhằm cải thiện hiệu quả so với các phương pháp kinh điển như Brute Force và các thuật toán dựa trên tiền xử lý đơn thuần. Ý tưởng chính của thuật toán là kết hợp giữa tiền xử lý mẫu và việc tận dụng thông tin đã thu được trong quá trình quét văn bản, từ đó giảm số lần so sánh ký tự không cần thiết và tối ưu hóa bước dịch cửa sổ tìm kiếm.

Thuật toán thực hiện so sánh ký tự theo hướng từ trái sang phải và đạt hiệu quả cao nhờ cơ chế tránh quay lui trên văn bản. Pha tiền xử lý và pha tìm kiếm đều được tối ưu để đảm bảo hiệu năng tuyến tính, đồng thời hạn chế tối đa số phép so sánh dư thừa trong quá trình xử lý chuỗi. Cụ thể, thuật toán có các đặc điểm sau:

- pha tiền xử lý có độ phức tạp thời gian và không gian $O(m)$;
- pha tìm kiếm có độ phức tạp thời gian $O(n)$;
- trong trường hợp xấu nhất, số phép so sánh ký tự trên văn bản bị chặn bởi $\frac{3}{2}n$.

3.7.2 Mô tả thuật toán

Thuật toán Apostolico–Crochemore sử dụng bảng dịch **kmpNext** (xem Chương 3.3, thuật toán Knuth–Morris–Pratt) để tính các bước dịch.

Gọi $\ell = 0$ nếu x là một lũy thừa của một ký tự đơn (tức là $x = c^m$ với $c \in \Sigma$), và ℓ bằng vị trí của ký tự đầu tiên trong x khác với $x[0]$ trong trường hợp còn lại (tức là $x = a^\ell bu$, với $a, b \in \Sigma$, $u \in \Sigma^*$ và $a \neq b$).

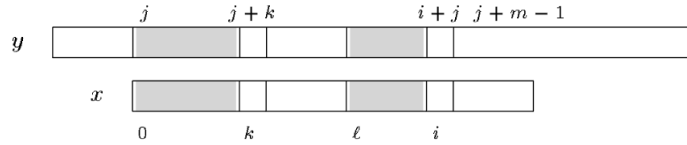
Trong mỗi lần thử, các phép so sánh được thực hiện theo thứ tự các vị trí trong mẫu như sau:

$$\ell, \ell + 1, \dots, m - 2, m - 1, 0, 1, \dots, \ell - 1.$$

Trong pha tìm kiếm, ta xét các bộ ba có dạng (i, j, k) trong đó:

- cửa sổ đang được đặt tại đoạn văn bản $y[j \dots j + m - 1]$;
- $0 \leq k \leq \ell$ và $x[0 \dots k - 1] = y[j \dots j + k - 1]$;
- $\ell \leq i < m$ và $x[\ell \dots i - 1] = y[j + \ell \dots j + i - 1]$.

Bộ ba khởi đầu là $(\ell, 0, 0)$.



Bây giờ ta giải thích cách tính bộ ba tiếp theo sau khi bộ ba (i, j, k) đã được tính. Ba trường hợp phát sinh tùy theo giá trị của i :

Trường hợp $i = \ell$:

- Nếu $x[i] = y[i + j]$ thì bộ ba tiếp theo là $(i + 1, j, k)$.
- Nếu $x[i] \neq y[i + j]$ thì bộ ba tiếp theo là $(\ell, j + 1, \max\{0, k - 1\})$.

Trường hợp $\ell < i < m$:

- Nếu $x[i] = y[i + j]$ thì bộ ba tiếp theo là $(i + 1, j, k)$.
- Nếu $x[i] \neq y[i + j]$ thì lại có hai trường hợp con tùy theo giá trị của $kmpNext[i]$:
 - $kmpNext[i] \leq \ell$: bộ ba tiếp theo là $(\ell, i + j - kmpNext[i], \max\{0, kmpNext[i]\})$.
 - $kmpNext[i] > \ell$: bộ ba tiếp theo là $(kmpNext[i], i + j - kmpNext[i], \ell)$.

Trường hợp $i = m$:

- Nếu $k < \ell$ và $x[k] = y[j + k]$ thì bộ ba tiếp theo là $(i, j, k + 1)$.
- Ngược lại — hoặc $k < \ell$ và $x[k] \neq y[j + k]$, hoặc $k = \ell$. Nếu $k = \ell$, một lần xuất hiện của x được ghi nhận. Trong cả hai trường hợp này, bộ ba tiếp theo được tính theo cách tương tự như trường hợp $\ell < i < m$.

Pha tiền xử lý bao gồm việc tính bảng *kmpNext* và số nguyên ℓ . Pha này có thể thực hiện trong $O(m)$ không gian và thời gian. Pha tìm kiếm có độ phức tạp thời gian $O(n)$; hơn nữa, thuật toán Apostolico–Crochemore thực hiện nhiều nhất $\frac{3}{2}n$ phép so sánh ký tự văn bản trong trường hợp xấu nhất.

3.7.3 Cài đặt bằng C++

```
#include <vector>
#include <string>
#include <algorithm>
#include <functional>

void preKmp(const std::string& x, int m, std::vector<int>& kmpNext) {
    int i = 0, j = kmpNext[0] = -1;
    while (i < m) {
        while (j > -1 && x[i] != x[j])
            j = kmpNext[j];
        kmpNext[++i] = ++j;
        if (x[i] == x[j])
            kmpNext[i] = kmpNext[j];
    }
}

std::vector<int> AXAMAC(const std::string& x, const std::string& y) {
    int m = static_cast<int>(x.size());
    int n = static_cast<int>(y.size());
    std::vector<int> kmpNext(m + 1);
    std::vector<int> results;

    /* Preprocessing */
    preKmp(x, m, kmpNext);
    int ell = 1;
    while (ell < m && x[ell - 1] == x[ell])
        ++ell;
    if (ell == m)
        ell = 0;
```

```

/* Searching */
int i = ell, j = 0, k = 0;
while (j <= n - m) {
    while (i < m && x[i] == y[i + j])
        ++i;
    if (i >= m) {
        while (k < ell && x[k] == y[j + k])
            ++k;
        if (k >= ell)
            results.push_back(j); // OUTPUT(j)
    }
    j += (i - kmpNext[i]);
    if (i == ell) {
        k = std::max(0, k - 1);
    } else if (kmpNext[i] <= ell) {
        k = std::max(0, kmpNext[i]);
        i = ell;
    } else {
        k = ell;
        i = kmpNext[i];
    }
}

return results;
}

```

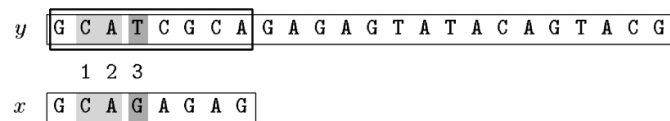
3.7.4 Kiểm nghiệm thuật toán

i	0	1	2	3	4	5	6	7	8
$x[i]$	G	C	A	G	A	G	A	G	
$kmpNext[i]$	-1	0	0	-1	1	-1	1	-1	1

$$\ell = 1$$

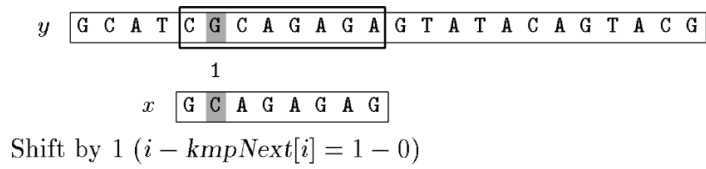
Pha tìm kiếm

Lần 1:

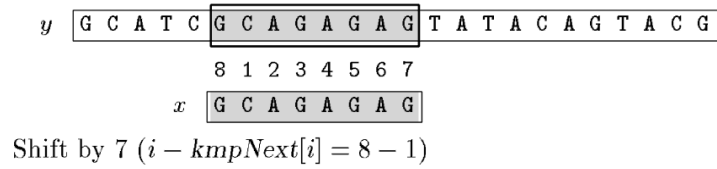


Shift by 4 ($i - kmpNext[i] = 3 - -1$)

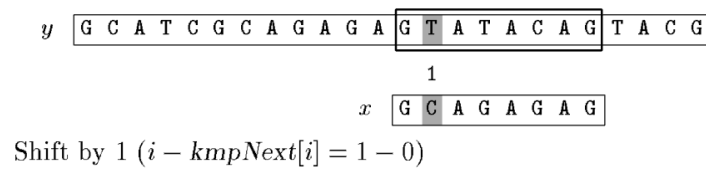
Lần 2:



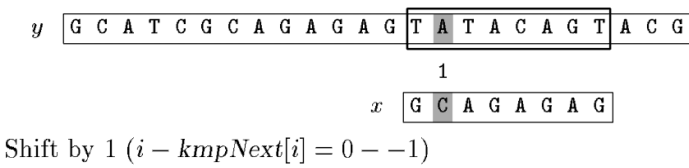
Lần 3:



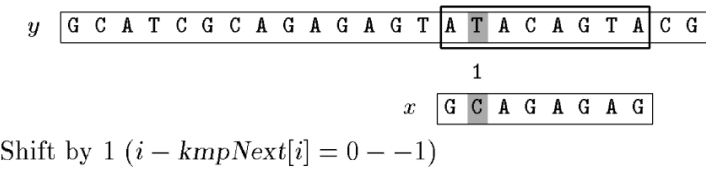
Lần 4:



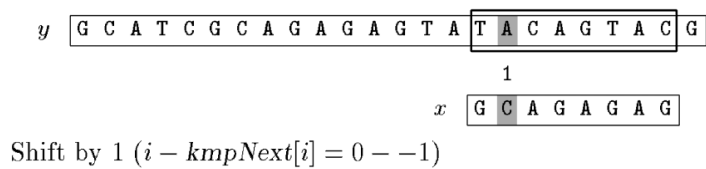
Lần 5:



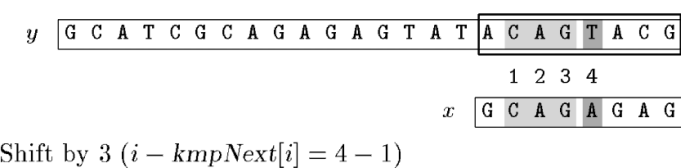
Lần 6:



Lần 7:



Lần 8:



Trong ví dụ trên, thuật toán Apostolico-Crochemore thực hiện 20 lần so sánh chuỗi.

3.8 Thuật toán Not So Naïve

3.8.1 Đặc điểm chính

Thuật toán Not So Naive là một cải tiến đơn giản so với thuật toán Naive trong bài toán tìm kiếm xâu. Thay vì so sánh lại toàn bộ mẫu tại mỗi vị trí, thuật toán sử dụng một nhận xét nhỏ về cấu trúc của mẫu để bỏ qua một số vị trí không cần thiết, giúp giảm số phép so sánh trong thực tế.

Thuật toán có các đặc điểm sau:

- Pha tiền xử lý thực hiện trong thời gian và không gian hằng số $O(1)$.
- Pha tìm kiếm có độ phức tạp thời gian $O(nm)$ trong trường hợp xấu nhất.
- Hiệu quả hơn thuật toán Naive theo hệ số trong trường hợp trung bình, tức là có tính chất *sub-linear* nhẹ theo hệ số.

3.8.2 Mô tả thuật toán

Trong pha tìm kiếm của thuật toán Not So Naive, các phép so sánh ký tự được thực hiện theo thứ tự vị trí của mẫu như sau: $1, 2, \dots, m-2, m-1, 0$.

Với mỗi lần thử, khi cửa sổ được đặt tại đoạn văn bản $y[j..j+m-1]$:

- Nếu $x[0] = x[1]$ và $x[1] \neq y[j+1]$, hoặc nếu $x[0] \neq x[1]$ và $x[1] = y[j+1]$, thì mẫu được dịch sang phải 2 vị trí sau lần thử đó.
- Trong các trường hợp còn lại, mẫu được dịch sang phải 1 vị trí.

Nhờ vậy, pha tiền xử lý có thể thực hiện trong thời gian và không gian hằng số $O(1)$. Pha tìm kiếm của thuật toán Not So Naive có độ phức tạp thời gian bậc hai $O(nm)$ trong trường hợp xấu nhất, nhưng có tính chất *sub-linear* nhẹ theo hệ số trong trường hợp trung bình.

3.8.3 Cài đặt bằng C++

```
#include <string>
#include <vector>
#include <cstring>

std::vector<int> NSN(const std::string& x, const std::string& y) {
    int m = static_cast<int>(x.size());
    int n = static_cast<int>(y.size());
    std::vector<int> results;

    /* Preprocessing */
```

```

int k, ell;
if (x[0] == x[1]) {
    k = 2;
    ell = 1;
} else {
    k = 1;
    ell = 2;
}

/* Searching */
int j = 0;
while (j <= n - m) {
    if (x[1] != y[j + 1]) {
        j += k;
    } else {
        if (std::memcmp(x.c_str() + 2, y.c_str() + j + 2, m - 2) == 0
            && x[0] == y[j])
            results.push_back(j); // OUTPUT(j)
        j += ell;
    }
}

return results;
}

```

3.8.4 Kiểm nghiệm thuật toán

Với $k = 1$ và $\ell = 2$

Lần 1:

y

G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 1 2 3
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

 Shift by 2

Lần 2:

y

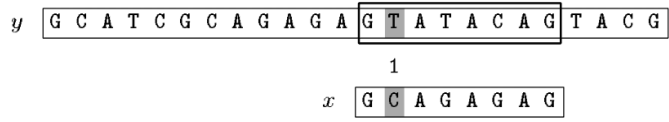
G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

 1
 x

G	C	A	G	A	G	A	G
---	---	---	---	---	---	---	---

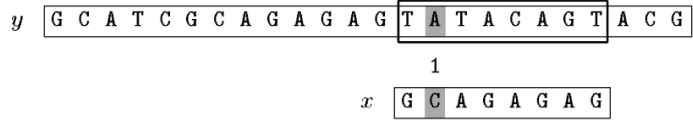
 Shift by 1

Lần 3:



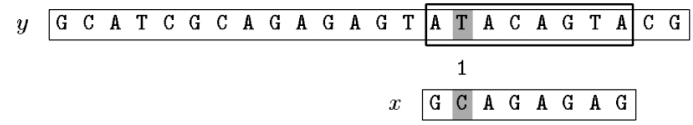
Shift by 1

Lần 11:



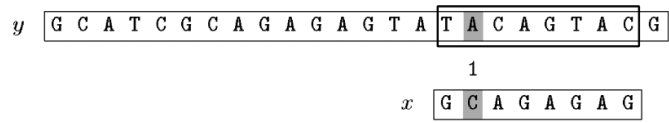
Shift by 1

Lần 12:



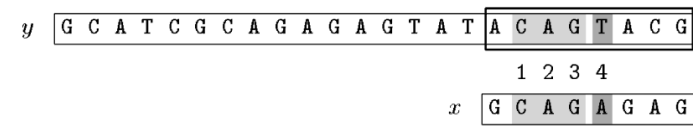
Shift by 1

Lần 13:



Shift by 1

Lần 14:



Shift by 2

Thuật toán Not So Naive thực hiện 27 lần so khớp chuỗi trong ví dụ trên.

Chương 4

TÌM KIẾM MẪU TỪ PHẢI SANG TRÁI

4.1 Thuật toán Boyer-Moore

Thuật toán tìm kiếm mẫu Boyer–Moore là một trong những thuật toán tìm kiếm chuỗi hiệu quả nhất và được xem như tiêu chuẩn đánh giá trong các bài toán đối sánh mẫu thực tế. Thuật toán này được phát triển bởi Robert Stephen Boyer và J Strother Moore vào năm 1977.

Thuật toán Boyer–Moore hoạt động bằng cách tiên xử lý xâu mẫu, sau đó quét văn bản từ phải sang trái, bắt đầu từ các ký tự ngoài cùng bên phải của mẫu. Thuật toán dựa trên nguyên tắc rằng nếu xảy ra sự không khớp, thì không cần phải kiểm tra các ký tự còn lại trong lần so khớp hiện tại. Cách tiếp cận theo hướng ngược này giúp giảm đáng kể độ phức tạp thời gian của thuật toán so với các phương pháp tìm kiếm chuỗi ngây thơ (naive).

Dưới đây là phần giải thích từng bước cách hoạt động của thuật toán Boyer–Moore:

4.1.1 Tiên xử lý

Thuật toán Boyer–Moore sử dụng hai heuristic (phương pháp kinh nghiệm) trong quá trình tiên xử lý mẫu:

- Heuristic ký tự sai (Bad Character Heuristic)
- Heuristic hậu tố tốt (Good Suffix Heuristic)

Heuristic ký tự sai (Bad Character Heuristic – xây dựng bảng ký tự sai)

Heuristic này xác định khoảng dịch chuyển của mẫu khi xảy ra tình huống không khớp ký tự.

Thuật toán xây dựng một mảng (thường gọi là “bảng ký tự sai”) để lưu vị trí xuất hiện gần nhất (bên phải nhất) của mỗi ký tự trong mẫu. Nếu một ký tự không xuất hiện trong mẫu, giá trị của nó được gán bằng độ dài của mẫu.

Đối với mỗi ký tự trong mẫu, bảng được cập nhật theo vị trí xuất hiện cuối cùng của ký tự đó trong mẫu với công thức:

$$BCT(ch) = \begin{cases} m & \text{if } ch \notin p \\ m - j - 1 & \text{otherwise} \end{cases}$$

Trong đó:

- m là độ dài của xâu mẫu p
- j là vị trí xuất hiện cuối cùng của ký tự ch trong xâu mẫu p
- ch là ký tự đang xét trong bảng chữ cái

Ví dụ:

	$j \rightarrow$	0	1	2	3	4	
P		M	I	S	S	I	Length(m)=5

Bad Character Table

M	I	S	*
4	0	1	5

$$\begin{aligned}
 \mathbf{M} &\rightarrow (m-j-1) = 5-0-1 = 4 \\
 \mathbf{I} &\rightarrow (m-j-1) = 5-1-1 = 3 \\
 \mathbf{S} &\rightarrow (m-j-1) = 5-2-1 = 2 \\
 \mathbf{S} &\rightarrow (m-j-1) = 5-3-1 = 1 \\
 \mathbf{I} &\rightarrow (m-j-1) = 5-4-1 = 0
 \end{aligned}$$

4.1.2 Tìm kiếm

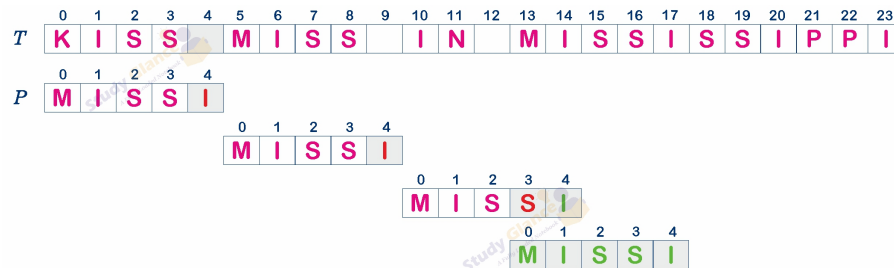
Sau khi quá trình tiền xử lý hoàn tất, thuật toán bắt đầu bước tìm kiếm thực sự:

- Căn chỉnh xâu mẫu với vị trí bắt đầu của văn bản.
- So sánh xâu mẫu với văn bản theo hướng từ phải sang trái.
- Nếu tất cả các ký tự trong xâu mẫu đều khớp với văn bản, thì một lần xuất hiện hợp lệ của mẫu đã được tìm thấy.

4.1.3 Dịch chuyển mẫu (Shifting the Pattern)

Khi xảy ra sự không khớp:

- Nếu ký tự gây ra không khớp có xuất hiện trong mẫu, dịch chuyển mẫu sao cho ký tự đó trong văn bản được căn chỉnh với lần xuất hiện phải nhất của nó trong mẫu.
- Nếu ký tự gây ra không khớp không xuất hiện trong mẫu, dịch chuyển toàn bộ mẫu vượt qua ký tự đó trong văn bản.



4.1.4 Lặp lại quá trình so khớp

Sau khi dịch chuyển mẫu theo các quy tắc đã nêu, thuật toán tiếp tục quá trình so sánh:

- Tiếp tục so sánh xâu mẫu với văn bản theo hướng từ phải sang trái.
- Mỗi khi xảy ra sự không khớp, áp dụng các quy tắc dịch chuyển tương ứng.
- Quá trình này được lặp lại cho đến khi duyệt hết văn bản hoặc tìm được tất cả các vị trí xuất hiện của mẫu.

4.1.5 Điều kiện kết thúc (Termination)

Thuật toán kết thúc khi xảy ra một trong hai trường hợp sau:

- Xâu mẫu đã được dịch chuyển vượt quá phần cuối của văn bản, nghĩa là không còn khả năng xuất hiện thêm kết quả khớp nào.
- Hoặc tất cả các lần xuất hiện của xâu mẫu trong văn bản đã được tìm thấy.

4.1.6 Cài đặt thuật toán bằng C++

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
```

```

#define NO_OF_CHARS 256

// Hàm xây dựng bảng Bad Character Heuristic
void buildBadCharTable(const string &pattern, vector<int> &badChar) {
    int m = pattern.size();

    // Khởi tạo tất cả giá trị = m
    for (int i = 0; i < NO_OF_CHARS; i++) {
        badChar[i] = m;
    }

    // Cập nhật vị trí xuất hiện cuối cùng
    for (int i = 0; i < m; i++) {
        badChar[(unsigned char)pattern[i]] = m - i - 1;
    }
}

// Hàm tìm kiếm Boyer-Moore (chỉ dùng Bad Character Heuristic)
vector<int> boyerMoore(string text, string pattern) {
    vector<int> result;

    int n = text.size();
    int m = pattern.size();

    vector<int> badChar(NO_OF_CHARS);

    buildBadCharTable(pattern, badChar);

    int shift = 0;

    while (shift <= n - m) {
        int j = m - 1;

        // So sánh từ phải sang trái
        while (j >= 0 && pattern[j] == text[shift + j]) {
            j--;
        }

        // Nếu khớp hoàn toàn

```

```

        if (j < 0) {
            result.push_back(shift);
            shift += (shift + m < n) ? m : 1;
        }
        else {
            // Dịch chuyển theo Bad Character Heuristic
            char bad = text[shift + j];
            shift += max(1, badChar[(unsigned char)bad] - (m - 1 - j));
        }
    }

    return result;
}

int main() {
    string text = "ababcbcabababd";
    string pattern = "ababd";

    vector<int> pos = boyerMoore(text, pattern);

    cout << "Vi tri xuat hien: ";
    for (int i : pos) {
        cout << i << " ";
    }

    return 0;
}

```

4.2 Thuật toán Turbo Boyer-Moore (Turbo-BM)

4.2.1 Đặc điểm chính

Turbo-Boyer-Moore (Turbo-BM) là một biến thể cải tiến của thuật toán Boyer-Moore, được đề xuất bởi Crochemore, Czumaj, Gasieniec, Lecroq, Plandowski và Rytter vào năm 1994. Thuật toán được thiết kế nhằm giảm số phép so sánh ký tự trong quá trình tìm kiếm chuỗi bằng cách tận dụng thông tin từ các lần so khớp trước đó để tăng khoảng dịch chuyển của mẫu.

Turbo-BM kế thừa toàn bộ các heuristics của Boyer-Moore như bad-character và good-suffix, đồng thời bổ sung cơ chế turbo-shift giúp cải thiện hiệu năng trong thực tế mà không làm tăng đáng kể chi phí bộ nhớ hay tiền xử lý.

Thuật toán có các đặc điểm sau:

- là một biến thể của thuật toán Boyer-Moore;
- không cần thêm bước tiền xử lý nào ngoài các bước của Boyer-Moore;
- chỉ cần thêm một lượng bộ nhớ hằng số so với Boyer-Moore;
- giai đoạn tiền xử lý có độ phức tạp thời gian và không gian $O(m + \sigma)$;
- giai đoạn tìm kiếm có độ phức tạp thời gian $O(n)$;
- trong trường hợp xấu nhất thực hiện không quá $2n$ phép so sánh;

4.2.2 Mô tả thuật toán

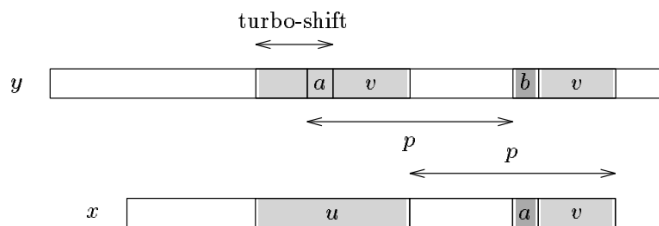
Thuật toán Turbo-BM là một cải tiến của thuật toán Boyer-Moore. Thuật toán không cần thêm bước tiền xử lý nào và chỉ yêu cầu một lượng bộ nhớ phụ hằng số so với thuật toán Boyer-Moore gốc. Ý tưởng của thuật toán là ghi nhớ đoạn của văn bản đã khớp với một hậu tố của mẫu trong lần thử trước đó (và chỉ khi thực hiện phép dịch chuyển good-suffix).

Kỹ thuật này mang lại hai lợi ích:

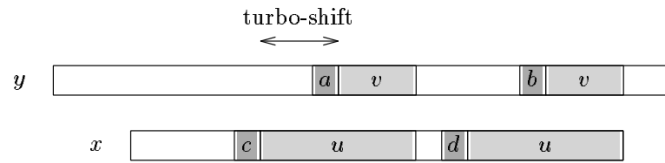
- có thể bỏ qua đoạn đã được ghi nhớ;
- có thể thực hiện phép dịch chuyển turbo-shift.

Turbo-shift có thể xảy ra khi trong lần thử hiện tại, hậu tố của mẫu khớp với văn bản ngắn hơn hậu tố đã được ghi nhớ từ lần thử trước đó. Trong trường hợp này, gọi u là đoạn đã được ghi nhớ và v là hậu tố khớp trong lần thử hiện tại sao cho uzv là một hậu tố của x . Gọi a và b lần lượt là các ký tự gây ra sự không khớp trong mẫu và văn bản ở lần thử hiện tại. Khi đó, av là một hậu tố của x , và do $|v| < |u|$, nên av cũng là hậu tố của u .

Hai ký tự a và b xuất hiện trong văn bản với khoảng cách p , và hậu tố của x có độ dài $|uzv|$ có chu kỳ độ dài $p = |zu|$ do u là biên của uzv . Vì vậy, hậu tố này không thể chồng lấp lên hai lần xuất hiện của hai ký tự khác nhau a và b ở khoảng cách p trong văn bản. Độ dịch chuyển nhỏ nhất có thể là $|u| - |v|$, và độ dịch chuyển này được gọi là turbo-shift.



Vẫn trong trường hợp $|v| - |u|$, nếu độ dài của phép dịch chuyển bad-character lớn hơn độ dài của phép dịch chuyển good-suffix và cả độ dài của turbo-shift, thì độ dài của phép dịch chuyển thực tế phải lớn hơn hoặc bằng $|u| + 1$. Thật vậy, trong trường hợp hai ký tự c và d là khác nhau, do ta giả sử rằng phép dịch chuyển trước đó là một phép dịch chuyển good-suffix.



Khi đó, một phép dịch chuyển lớn hơn turbo-shift nhưng nhỏ hơn $|u| + 1$ sẽ làm cho hai ký tự c và d được căn chỉnh với cùng một ký tự trong v . Do đó, trong trường hợp này, độ dài của phép dịch chuyển thực tế phải lớn hơn hoặc bằng $|u| + 1$.

Giai đoạn tiền xử lý của thuật toán có thể được thực hiện với độ phức tạp thời gian và không gian $O(m + \sigma)$. Giai đoạn tìm kiếm có độ phức tạp thời gian $O(n)$. Số phép so sánh ký tự trên văn bản mà thuật toán Turbo-Bm thực hiện bị chặn trên bởi $2n$.

4.2.3 Cài đặt bằng C++

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

const int ASIZE = 256;

/* Bad Character Heuristic */
void preBmBc(const string &pat, vector<int> &bmBc) {
    int m = pat.length();

    bmBc.assign(ASIZE, m);

    for (int i = 0; i < m - 1; ++i) {
        bmBc[(unsigned char)pat[i]] = m - 1 - i;
    }
}

/* Suffixes */
```

```

void suffixes(const string &pat, vector<int> &suff) {
    int m = pat.length();

    suff.resize(m);
    suff[m - 1] = m;

    int g = m - 1;
    int f = 0;

    for (int i = m - 2; i >= 0; --i) {
        if (i > g && suff[i + m - 1 - f] < i - g) {
            suff[i] = suff[i + m - 1 - f];
        } else {
            if (i < g)
                g = i;

            f = i;

            while (g >= 0 && pat[g] == pat[g + m - 1 - f])
                --g;

            suff[i] = f - g;
        }
    }
}

/* Good Suffix Heuristic */
void preBmGs(const string &pat, vector<int> &bmGs) {
    int m = pat.length();

    vector<int> suff(m);

    suffixes(pat, suff);

    bmGs.assign(m, m);

    int j = 0;

    for (int i = m - 1; i >= 0; --i) {
        if (suff[i] == i + 1) {

```

```

        for (; j < m - 1 - i; ++j) {
            if (bmGs[j] == m)
                bmGs[j] = m - 1 - i;
        }
    }

    for (int i = 0; i <= m - 2; ++i) {
        bmGs[m - 1 - suff[i]] = m - 1 - i;
    }
}

/* Turbo Boyer-Moore */
void TBM(const string &pat, const string &txt) {
    int m = pat.length();
    int n = txt.length();

    vector<int> bmBc;
    vector<int> bmGs;

    /* Preprocessing */
    preBmBc(pat, bmBc);
    preBmGs(pat, bmGs);

    /* Searching */
    int j = 0;
    int u = 0;
    int shift = m;

    while (j <= n - m) {
        int i = m - 1;

        while (i >= 0 && pat[i] == txt[i + j]) {
            --i;

            if (u != 0 && i == m - 1 - shift)
                i -= u;
        }

        if (i < 0) {

```

```

        cout << "Pattern found at index: " << j << endl;

        shift = bmGs[0];
        u = m - shift;
    } else {
        int v = m - 1 - i;
        int turboShift = u - v;

        int bcShift =
            bmBc[(unsigned char)txt[i + j]] - m + 1 + i;

        shift = max(turboShift, bcShift);
        shift = max(shift, bmGs[i]);

        if (shift == bmGs[i]) {
            u = min(m - shift, v);
        } else {
            if (turboShift < bcShift)
                shift = max(shift, u + 1);

            u = 0;
        }
    }

    j += shift;
}

int main() {
    string text = "ABAAAABCDABCABC";
    string pattern = "ABC";

    TBM(pattern, text);

    return 0;
}

```

4.2.4 Kiểm nghiệm thuật toán

a	A	C	G	T
$bmBc[a]$	1	6	2	8

i	0	1	2	3	4	5	6	7
$x[i]$	G	C	A	G	A	G	A	G
$suff[i]$	1	0	0	2	0	4	0	8
$bmGs[i]$	7	7	7	2	7	4	7	1

Pha tìm kiếm:

Lần 1:

First attempt

G C A T C G C **A** G A G A G T A T A C A G T A C G

1

G C A G A G A G

Shift by: 1 ($bmGs[7]=bmBc[A]-8+8$)

Lần 2:

Second attempt

G C A T C G C A G A G A G T A T A C A G T A C G

3 2 1

G C A G A G A G

Shift by: 4 ($bmGs[5]=bmBc[C]-8+6$)

Lần 3:

Third attempt

G C A T C G C A G A G A G T A T A C A G T A C G

6 5 - - 4 3 2 1

G C A G A G A G

Shift by: 7 (*bmGs*[0])

Lần 4:

Fourth attempt

G C A T C G C A G A G A G T A T A C A G T A C G

3 2 1

G C A G A G A G

Shift by: 4 ($bmGs[5]=bmBc[C]-8+6$)

Lần 5:

Fifth attempt

G C A T C G C A G A G A G T A T A C A G T A **C** G
 2 1
 G C A G A G **A** G

Shift by: 7 (*bmGs*[6])

Thuật toán Turbo-BM thực hiện 15 lần so sánh chuỗi trong ví dụ trên.

4.3 Thuật toán Quick Search

4.3.1 Đặc điểm chính

Quick Search là một thuật toán tìm kiếm chuỗi được đề xuất bởi Daniel M. Sunday như một biến thể đơn giản hóa của thuật toán Boyer-Moore. Thuật toán chỉ sử dụng heuristic bad-character để xác định khoảng dịch chuyển của mẫu, nhờ đó cấu trúc cài đặt trở nên ngắn gọn và dễ triển khai hơn so với Boyer-Moore đầy đủ.

Mặc dù có độ phức tạp thời gian trong trường hợp xấu nhất là bậc hai, Quick Search thường cho hiệu năng rất tốt trong thực tế, đặc biệt đối với các mẫu ngắn và bảng chữ cái lớn. Nhờ cơ chế dịch chuyển dựa trên ký tự ngay sau cửa sổ hiện tại, thuật toán thường đạt được khoảng dịch chuyển lớn và giảm số lần thử khớp không cần thiết.

Thuật toán có các đặc điểm sau:

- là phiên bản đơn giản hóa của thuật toán Boyer-Moore;
- chỉ sử dụng phép dịch chuyển bad-character;
- dễ cài đặt;
- giai đoạn tiền xử lý có độ phức tạp thời gian $O(m + \sigma)$ và độ phức tạp không gian $O(\sigma)$;
- giai đoạn tìm kiếm có độ phức tạp thời gian $O(mn)$;
- hoạt động rất nhanh trong thực tế đối với mẫu ngắn và bảng chữ cái lớn.

4.3.2 Mô tả thuật toán

Thuật toán Quick Search chỉ sử dụng bảng dịch chuyển bad-character (xem thuật toán Boyer-Moore). Sau mỗi lần thử mà cửa sổ được đặt trên đoạn văn bản $y[j \dots j+m-1]$, độ dài dịch chuyển luôn lớn hơn hoặc bằng 1. Do đó, ký tự $y[j+m]$ chắc chắn sẽ tham gia vào lần thử kế tiếp, vì vậy ký tự này có thể được sử dụng để tính toán phép dịch chuyển bad-character cho lần thử hiện tại.

Phép dịch chuyển bad-character trong thuật toán này được điều chỉnh nhẹ để xét thêm ký tự cuối cùng của mẫu x như sau: với mọi ký tự $cin\Sigma$,

$$qsBc[c] = \begin{cases} \min\{i \mid 0 < i \leq m \text{ và } x[m-i] = c\}, & \text{nếu } c \text{ xuất hiện trong } x; \\ m+1, & \text{ngược lại.} \end{cases}$$